

DocMind AI — Complete Project Documentation

Private document — do not commit to version control. This file is listed in `.gitignore` and will not be pushed to GitHub.

Table of Contents

1. [Project Overview](#)
 2. [Technology Stack](#)
 3. [File and Folder Structure](#)
 4. [Core Files — Detailed Reference](#)
 - 4.1 [Backend Files](#)
 - 4.2 [Service Layer](#)
 - 4.3 [Database Models](#)
 - 4.4 [Route Blueprints](#)
 - 4.5 [Frontend — Templates](#)
 - 4.6 [Frontend — JavaScript](#)
 - 4.7 [Frontend — CSS](#)
 - 4.8 [Deployment Files](#)
 5. [IBM Cloud Platform — Usage & Application](#)
 6. [Scalability and Innovativeness](#)
 7. [Contribution and Social Benefit](#)
 8. [Readiness for Deployment](#)
 9. [Commercial and Market Viability](#)
 10. [Future Scope](#)
 11. [Environment Variables Reference](#)
 12. [API Endpoints Reference](#)
 13. [Security Architecture](#)
-

1. Project Overview

DocMind AI is a full-stack, AI-powered intelligent document workspace built with Python Flask and IBM watsonx.ai foundation models. Users can upload documents in multiple formats and interact with them through natural language — asking questions, generating summaries, comparing two documents, and extracting AI-powered insights — all from a clean, responsive, enterprise-

grade web interface.

Problem Statement

In modern workplaces, students, researchers, legal teams, HR departments, and business analysts regularly deal with large volumes of documents — reports, contracts, research papers, presentations, and manuals. Reading and understanding all of them is time-consuming and inefficient. There is a clear need for an AI assistant that can:

- Instantly understand any uploaded document
- Answer questions about the document in plain language
- Summarise long documents in multiple formats
- Compare two documents and highlight differences
- Automatically extract keywords, topics, and tags

DocMind AI solves exactly this problem using IBM's enterprise-grade AI infrastructure.

Key Highlights

Property	Value
Language	Python 3.10+
Framework	Flask 3.0.3
AI Backend	IBM watsonx.ai (Llama 3.3 70B Instruct)
Database	SQLite (dev) / PostgreSQL (production)
Authentication	Flask-Login + Werkzeug password hashing
Frontend	Bootstrap 5 + Custom CSS design system
Deployment	Render, Railway, Heroku, Vercel
Document Formats	PDF, DOCX, PPTX, TXT
Max File Size	16 MB
Total Routes	28
Total Source Files	30+

2. Technology Stack

Backend

Technology	Version	Purpose
Python	3.10+	Core programming language
Flask	3.0.3	Web framework — routing, templating, request handling
Flask-SQLAlchemy	3.1.1	ORM — database models and

Technology	Version	Purpose
		queries
Flask-Login	0.6.3	User session management, login/logout, @login_required
Flask-WTF	1.2.1	CSRF protection and form utilities
Flask-Limiter	3.8.0	Rate limiting on auth routes (20/hr login, 10/hr register)
Werkzeug	3.0.3	Password hashing (PBKDF2-SHA256), file serving, proxy fix
python-dotenv	1.0.1	Loads .env file into os.environ at startup
SQLite / PostgreSQL	—	Relational database for users, documents, conversations
Gunicorn	22.0+	WSGI production server (Linux/cloud only)

IBM AI Platform

Technology	Version	Purpose
ibm-watsonx-ai	≥ 1.1.2 (tested 1.5.x)	Official Python SDK for watsonx.ai API
ibm-cloud-sdk-core	≥ 3.22.0	IBM Cloud authentication and HTTP client
Llama 3.3 70B Instruct	Meta (via watsonx)	Foundation model for all AI features
IBM Cloud IAM	—	API key-based authentication to watsonx.ai

Document Parsing

Technology	Version	Purpose
pymupdf (fitz)	≥ 1.24.0	PDF text extraction (pre-built wheel, no compiler needed)
python-docx	1.1.2	Microsoft Word (.docx) parsing
python-pptx	0.6.23	PowerPoint (.pptx) text extraction from all slides/shapes
Built-in open ()	—	Plain text (.txt) file reading with UTF-8 fallback

Frontend

Technology	Version	Purpose
Bootstrap 5	5.3.3 (CDN)	Responsive grid, components, utilities
Bootstrap Icons	1.11.3 (CDN)	Icon library (200+ icons used throughout)
Jinja2	(bundled with Flask)	Server-side HTML templating
Vanilla JavaScript	ES2020	All interactivity — no jQuery, no React
CSS Custom Properties	—	Full design system with light/dark theming

Deployment & DevOps

Technology	Purpose
Render	Primary cloud hosting platform
Railway	Alternative cloud hosting
Heroku	Alternative cloud hosting
Vercel	Serverless deployment option
Procfile	Gunicorn start command for all PaaS platforms
Git / GitHub	Version control and CI/CD trigger

3. File and Folder Structure

docmind_ai/ root)	← Project root (also the git repository)
├── run.py	← Local development server entry point
├── wsgi.py	← Production WSGI entry point (used by gunicorn)
├── config.py	← All application settings +
AGENT_CONFIGURATION	
├── requirements.txt	← All Python dependencies with exact versions
├── Procfile	← gunicorn start command
(Heroku/Railway/Render)	
├── railway.toml	← Railway deployment configuration
├── render.yaml	← Render blueprint (one-click deploy spec)
├── vercel.json	← Vercel serverless configuration
├── .env	← Real secrets (NEVER committed –
in .gitignore)	
├── .env.example	← Template for .env showing all required variables
├── .gitignore	← Excludes venv/, .env, __pycache__, uploads/

.gitattributes	← Enforces LF line endings (cross-platform)
README.md	← Public project documentation
DOCUMENTATION.md	← This file (private – not pushed to GitHub)
uploads/	← Uploaded files stored here (gitignored)
instance/	← SQLite database file stored here
(gitignored)	
api/	
index.py	← Vercel serverless adapter
app/	
__init__.py	← Main application package
extension init	← Application factory (create_app),
models/	
__init__.py	← Re-exports all models
models.py	← User, Document, Conversation, Message
models	
routes/	
__init__.py	← Re-exports all blueprints
misc.py	← Landing page (/), health check (/health)
auth.py	← /auth/register, /auth/login, /auth/logout
dashboard.py	← /dashboard/ (stats + recent items)
documents.py	← /documents/* (upload, view, delete,
download)	
ai.py	← /ai/* (chat, summary, compare, insights)
profile.py	← /profile/* (settings, password, delete
account)	
services/	
__init__.py	← Re-exports all service functions
watsonx_service.py	← IBM watsonx.ai integration layer
document_service.py	← Document text extraction
(PDF/DOCX/PPTX/TXT)	
static/	
css/	
main.css	← Full design system (light + dark mode)
dark.css	← Intentionally empty (dark mode is in
main.css)	
js/	
main.js	← Global: theme toggle, sidebar, toast,
formatAIText	
chat.js	← AI Chat page interactivity
summary.js	← AI Summary page interactivity
compare.js	← Document Comparison page interactivity
insights.js	← AI Insights page interactivity
documents.js	← Drag-and-drop upload zone
img/	← Static images (empty by default)
templates/	
base.html	← Master layout: sidebar, topbar, flash
messages	
landing.html	← Public marketing/home page (no login
required)	
auth/	
login.html	← Login form
register.html	← Registration form
dashboard/	
index.html	← Dashboard with stats and recent items

search	documents/	
	└─ index.html	← Document library with drag-drop upload +
	└─ view.html	← Individual document preview and actions
	ai/	
(short/detailed/bullets)	└─ chat.html	← ChatGPT-style conversational AI interface
	└─ summary.html	← Summary generator
	└─ compare.html	← Side-by-side document comparison
account	└─ insights.html	← Keywords, topics, tags, reading time
	profile/	
	└─ index.html	← Account settings, change password, delete
	errors/	
	└─ 404.html	← Not Found error page
	└─ 500.html	← Internal Server Error page
	└─ 429.html	← Rate Limit Exceeded page

4. Core Files — Detailed Reference

4.1 Backend Files

run.py — Local Development Entry Point

Purpose: Start the Flask development server for local use
 Run with: `python run.py`
 Port: 5000 (configurable via `FLASK_PORT` env var)

Key logic:

- Injects the project root AND the venv's `site-packages` into `sys.path` at startup
- This prevents the Flask reloader from losing the virtual environment when it restarts the process (a common issue on Windows with Python 3.12+)
- Calls `create_app()` and runs with `debug=True` and `use_reloader=True` in development

wsgi.py — Production Entry Point

Purpose: WSGI application object for gunicorn
 Used by: `gunicorn wsgi:application --bind 0.0.0.0:$PORT`

- Exposes the Flask app as both `application` (gunicorn default) and `app` (some platforms)
- Adds project root to `sys.path` so imports resolve correctly in production containers

config.py — Application Configuration

Purpose: Centralised settings for all environments
 Pattern: Class-based configuration hierarchy

Configuration classes:

Class	Used when	Debug	DB
DevelopmentConfig	FLASK_ENV=development	True	SQLite (local instance/)
ProductionConfig	FLASK_ENV=production	False	From DATABASE_URL env var
TestingConfig	FLASK_ENV=testing	True	SQLite in-memory

AGENT_CONFIGURATION dict — single place to tune all AI behaviour:

Key	Type	Default	Effect
model_id	string	meta-llama/llama-3-3-70b-instruct	Which watsonx model to call
personality	string	Expert document analyst persona	Injected into every prompt as system role
tone	string	professional	Appended as tone instruction
max_new_tokens	int	1024	Hard cap on generated response length
temperature	float	0.7	Creativity (0=deterministic, 1=random)
top_p	float	0.9	Nucleus sampling threshold
repetition_penalty	float	1.1	Discourages repeating phrases
summary_style	string	bullets	Default summary format
supported_languages	list	5 languages	Informational; guides multilingual use
safety_instructions	string	Content policy	Prepended to every AI request

app/__init__.py — Application Factory

Pattern: Flask Application Factory (create_app())
 Purpose: Creates and configures the Flask app, registers all extensions and blueprints

Initialisation sequence:

1. Create Flask app with template and static folder paths
2. Load configuration from config.py via get_config()

3. Apply `ProxyFix` middleware (trusts `X-Forwarded-*` headers from Render/Railway)
 4. Initialise `SQLAlchemy`, `Flask-Login`, `Flask-Limiter`
 5. Create upload folder and instance folder if they don't exist
 6. Register all 6 blueprints with their URL prefixes
 7. Register error handlers for 404, 500, 429
 8. Call `db.create_all()` inside app context to create DB tables
-

4.2 Service Layer

app/services/watsonx_service.py

This is the **core AI integration file**. It handles all communication with IBM watsonx.ai.

`_get_client()`

- Reads `IBM_API_KEY`, `IBM_PROJECT_ID`, and `IBM_WATSONX_URL` from config
- Creates an `IBM Credentials` object and a `ModelInference` instance
- Returns a configured model client ready to generate text
- Compatible with `ibm-watsonx-ai` $\geq 1.1.2$ (handles both old `MetaNames` and new `TextGenParameters` API)

`_build_system_preamble()`

- Assembles the system prompt from `AGENT_CONFIGURATION`
- Combines safety instructions + personality + tone into one string
- This string is prepended to every single AI prompt

`_call(prompt)`

- Sends prompt to model via `model.generate_text()`
- Returns stripped text or an error string starting with `[AI Error]`
- Logs errors via Flask's logger without crashing the request

`answer_question(document_text, question, history)`

- Builds a context-aware prompt including the last 6 conversation turns
- Limits document text to 6,000 characters to stay within context window
- Returns the AI's answer as a plain string

`summarise_document(document_text, style)`

- Supports three summary styles: `short`, `detailed`, `bullets`
- Each style has a different instruction injected into the prompt

- Limits document text to 7,000 characters

compare_documents(text_a, text_b, name_a, name_b)

- Sends both documents (truncated to 3,500 chars each) in one prompt
- Asks the model to return four clearly labelled sections
- Parses the response with regex into a structured dict: `similarities`, `differences`, `key_topics`, `summary`

extract_insights(document_text)

- Extracts KEYWORDS, TOPICS, TAGS, ONE_LINE_SUMMARY from the document
- Uses line-based regex parsing to extract each labelled field
- Returns a dict of lists (keywords, topics, tags) and a string (summary)

app/services/document_service.py

extract_text(file_path)

- Routes to the correct extractor based on file extension
- Returns a tuple: (text: str, page_count: int)

_extract_pdf(file_path)

- Uses PyMuPDF (`fitz`) to open and read PDF pages
- Joins all pages with newline separators

_extract_docx(file_path)

- Uses `python-docx` to read all non-empty paragraphs

_extract_pptx(file_path)

- Uses `python-pptx` to iterate all slides and shapes
- Extracts text from any shape that has a `.text` attribute

_extract_txt(file_path)

- Reads with UTF-8 encoding and `errors="replace"` for resilience

count_words(text) / estimate_reading_time(word_count)

- Simple word count and reading time estimate (200 WPM default)

4.3 Database Models

File: `app/models/models.py`

User

Table: `users`

Fields: `id`, `username` (unique), `email` (unique), `password_hash`, `created_at`, `theme`

- Inherits `UserMixin` (Flask-Login interface: `is_authenticated`, `is_active`, etc.)
- `set_password()` uses `generate_password_hash` (PBKDF2-SHA256 algorithm, Werkzeug)
- `check_password()` uses constant-time comparison to prevent timing attacks
- `theme` field stores "light" or "dark" — persisted per user in the database
- Has `documents` and `conversations` relationships with cascade delete

Document

Table: `documents`

Fields: `id`, `user_id` (FK), `original_name`, `stored_name` (UUID), `file_type`, `file_size`, `page_count`, `word_count`, `reading_time`, `content_text`, `ai_tags`, `ai_summary`, `uploaded_at`

- `stored_name` is a UUID hex string (prevents filename collisions and path traversal)
- `content_text` stores the full extracted text — this is what the AI reads
- `ai_tags` and `ai_summary` are populated after AI analysis and cached
- `size_human` property formats bytes into B/KB/MB/GB automatically

Conversation

Table: `conversations`

Fields: `id`, `user_id` (FK), `title` (first 80 chars of first message), `created_at`, `updated_at`

- Each conversation belongs to one user
- Title is set from the first message sent in that conversation
- `updated_at` is refreshed on every new message

Message

Table: `messages`

Fields: `id`, `conversation_id` (FK), `document_id` (FK, nullable), `role`, `content`, `created_at`

- `role` is either "user" or "assistant"
- Links to both the conversation and the specific document it references
- Ordered by `created_at` for chronological display

4.4 Route Blueprints

routes/misc.py — Prefix: none

Route	Method	Auth	Description
/	GET	No	Landing page (redirects)

Route	Method	Auth	Description
			to dashboard if logged in)
/health	GET	No	Database ping health check for hosting platforms

routes/auth.py — Prefix: /auth

Route	Method	Auth	Rate Limit	Description
/auth/register	GET, POST	No	10/hour	Register new account
/auth/login	GET, POST	No	20/hour	Login with email + password
/auth/logout	GET	Yes	—	Clear session and redirect
/auth/toggle-theme	POST	Yes	—	Flip user theme (light ↔ dark), saves to DB

routes/dashboard.py — Prefix: /dashboard

Route	Method	Auth	Description
/dashboard/	GET	Yes	Stats (total docs, words, pages), recent docs, recent conversations

routes/documents.py — Prefix: /documents

Route	Method	Auth	Description
/documents/	GET	Yes	Document library with optional ?q= search
/documents/upload	POST	Yes	Save file, extract text, create Document record
/documents/<id>	GET	Yes	View document details and content preview
/documents/<id>/delete	POST	Yes	Delete file from disk and DB record
/documents/download/<id>	GET	Yes	Serve original file as attachment
/documents/api/list	GET	Yes	JSON list of all documents (used by AI

Route	Method	Auth	Description
			page dropdowns)

routes/ai.py — Prefix: /ai

Route	Method	Auth	Description
/ai/chat	GET	Yes	Chat interface page
/ai/chat/send	POST (JSON)	Yes	Send message, get AI response, save to DB
/ai/chat/new-conversation	POST	Yes	Create blank conversation
/ai/chat/delete-conversation/<id>	POST	Yes	Delete conversation + all its messages
/ai/summary	GET	Yes	Summary generator page
/ai/summary/generate	POST (JSON)	Yes	Generate summary, cache to Document.ai_summary
/ai/compare	GET	Yes	Document comparison page
/ai/compare/run	POST (JSON)	Yes	Run comparison between two documents
/ai/insights	GET	Yes	Insights page (can pre-load with ?doc_id=)
/ai/insights/generate	POST (JSON)	Yes	Extract insights, cache tags to Document.ai_tags

routes/profile.py — Prefix: /profile

Route	Method	Auth	Description
/profile/	GET	Yes	Account settings page
/profile/update	POST	Yes	Change username
/profile/change-password	POST	Yes	Change password (requires current password)
/profile/delete-account	POST	Yes	Delete user, all files, all data

4.5 Frontend — Templates

base.html

The master layout used by all authenticated pages.

- Sets `data-theme` attribute from `current_user.theme` (drives CSS variable switching)
- Renders the full sidebar with 8 navigation links grouped into sections
- Renders the topbar with page title, theme toggle button, and Upload shortcut
- Handles flash message display with auto-dismiss (5 seconds)
- Includes Bootstrap 5 JS and `main.js` at the bottom of every page
- Uses `{% block content %}` for authenticated users and `{% block auth_content %}` for auth/error pages

landing.html

Standalone page (does not extend `base.html`) — has its own full HTML structure.

- Sticky navigation bar
- Hero section with animated chat preview card
- Stats strip (4 numbers)
- 6-card features grid
- 3-step "How it works" section
- CTA section
- Footer

ai/chat.html

- Two-column layout: conversation sidebar + chat main area
- `window.CHAT_DATA` object passes server-side data (URLs, IDs) to JavaScript
- Quick prompt buttons populate the textarea on click
- Keyboard shortcuts: `Enter` to send, `Shift+Enter` for newline
- Thinking dots animation shown while AI is processing

ai/summary.html

- Two-column layout: settings panel (left) + output panel (right)
- Three radio-button style pickers with `:has(input:checked)` CSS highlight
- Copy-to-clipboard button appears after generation

ai/compare.html

- Document selectors with "VS" divider
- Results displayed in a 2×2 card grid (Similarities, Differences, Key Topics, Summary)

ai/insights.html

- 4 stat cards for Pages, Words, Reading Time, AI Tags count
 - Badge clouds for Keywords, Topics, Category Tags
 - One-line summary card
-

4.6 Frontend — JavaScript

main.js — Global utilities (loaded on every page)

- **Theme toggle:** Reads data-theme from `<html>`, swaps it, fires a POST to `/auth/toggle-theme` to persist to DB
- **Sidebar toggle:** Opens/closes sidebar on mobile via `.open` class; closes on outside click
- **Alert auto-dismiss:** All `.alert` elements auto-close after 5 seconds
- **showToast(message, type):** Global function that creates Bootstrap toast notifications, appends to `#toastContainer`, auto-removes after 4 seconds
- **formatAIText(text):** Converts AI markdown-like output to safe HTML — handles `**bold**`, `#` headings, bullet lists (`- * •`), numbered lists, paragraph breaks

chat.js

- Manages `convId` and `docId` state between page load and user interactions
- `sendMessage()`: posts to `/ai/chat/send`, renders user + assistant bubbles, updates conversation label
- `appendBubble(role, content)`: creates bubble DOM element with timestamp
- Auto-scrolls to bottom after each new message
- New conversation button clears the chat area

summary.js

- Reads selected document ID and summary style radio button
- Shows/hides loading spinner, empty state, result panels
- Copy button uses `navigator.clipboard.writeText()`

compare.js

- Validates both document selects are filled and different

- Posts to `/ai/compare/run`, fills four result sections with `formatAIText()`

insights.js

- `generateInsights()` posts to `/ai/insights/generate`
- `renderBadges(containerId, items, cssClass)` creates badge elements for keywords/topics/tags
- If `cfg.docId` is pre-set from the server, shows results immediately on page load

documents.js

- Drag-and-drop: `dragenter/dragover` add `.drag-over` class, `drop` sets files
 - File validation: checks extension and size client-side before submitting
 - Progress bar: fake progress ticker (to 90%) then snaps to 100% on form submit
 - Double-click fix: `e.target.closest("button")` prevents the zone click handler from firing when the Browse button is clicked
-

4.7 Frontend — CSS (`main.css`)

The entire design system lives in one file. No external CSS frameworks are customised — only Bootstrap's defaults are selectively overridden.

Architecture:

- **CSS Custom Properties** (`--bg`, `--surface`, `--accent`, etc.) define all colours and spacing
- Light mode values are on `:root`, dark mode values override them on `[data-theme="dark"]`
- Switching theme is done purely by toggling `data-theme` on `<html>` — no class swaps, no stylesheet injection

Key sections:

- `:root` and `[data-theme="dark"]` — 30+ CSS variables
- `.glass-card` — the universal card component (white surface, border, subtle shadow)
- `.sidebar` + `.main-wrapper` — fixed sidebar + scrollable content area layout
- `.stat-card` — icon + value + label row layout for dashboard numbers
- `.chat-layout` — flex container that fills viewport height for the chat interface
- `.chat-bubble-wrap` — two-direction bubble layout (user right, assistant left)
- `.thinking-overlay` — animated 3-dot loading indicator
- `.prose` — styles for AI text output (lists, bold, line-height)

- `.upload-zone` — dashed border with hover/drag highlight state
- `.doc-type-badge` — colour-coded file type pill (PDF=red, DOCX=blue, PPTX=orange, TXT=gray)
- Bootstrap overrides for buttons, forms, dropdowns, alerts, modals (all themed)

4.8 Deployment Files

Procfile

```
web: gunicorn wsgi:application --bind 0.0.0.0:$PORT --workers 2 --timeout 120 --preload
```

Used by Heroku, Railway, and Render as the process startup command.

render.yaml

One-click Render blueprint. Specifies:

- Runtime: Python
- Build command: `pip install -r requirements.txt`
- Start command: `gunicorn`
- Health check path: `/health`
- Environment variables including `DATABASE_URL=sqlite:///tmp/docmind.db`

railway.toml

Railway-specific config. Uses Nixpacks builder, same gunicorn command.

vercel.json + api/index.py

Serverless deployment adapter. Wraps the Flask app in a Vercel-compatible handler.

5. IBM Cloud Platform — Usage & Application

How IBM watsonx.ai is Used

DocMind AI uses IBM watsonx.ai as its **sole AI inference engine**. Every intelligent feature in the application calls the watsonx.ai API:

```
User Action → Flask Route → watsonx_service.py → IBM watsonx.ai API → Llama 3.3 70B → Response
```

Authentication Flow

```
IBM Cloud Account
├── IAM API Key (IBM_API_KEY)
│   └── ibm_watsonx_ai.Credentials(url=IBM_WATSONX_URL,
api_key=IBM_API_KEY)
```



```
└─ ModelInference(model_id=..., project_id=IBM_PROJECT_ID)
   └─ model.generate_text(prompt=...)
```

- The API key never leaves the server — it lives in `.env` and is accessed only by `watsonx_service.py`
- The project ID ties the requests to the user's specific watsonx.ai project on IBM Cloud
- The region URL (`eu-gb.ml.cloud.ibm.com` for EU London) routes requests to the nearest IBM data center

Model Configuration

The model used is **meta-llama/llama-3-3-70b-instruct** — a 70-billion parameter instruction-tuned language model hosted on IBM's watsonx.ai infrastructure. Key generation parameters:

```
max_new_tokens: 1024 (response length cap)
temperature:    0.7   (balanced creativity)
top_p:          0.9   (nucleus sampling)
repetition_penalty: 1.1 (reduces repetitive output)
```

Why IBM watsonx.ai is the Appropriate Choice

Requirement	Why IBM watsonx.ai Fits
Enterprise document understanding	watsonx.ai is purpose-built for enterprise knowledge work — not consumer chat
Privacy and data sovereignty	IBM allows regional deployment (EU, US, JP) — documents stay in chosen geography
Reliable API with SLAs	IBM Cloud provides enterprise-grade uptime, not best-effort consumer APIs
Foundation model flexibility	Project can switch from Llama to IBM's own Granite models with one config change
Responsible AI	IBM's AI Ethics principles are embedded in watsonx — safety guardrails are built-in
Security compliance	IBM Cloud is GDPR-compliant, ISO-certified — critical for enterprise document handling

Prompt Engineering Strategy

Every AI call uses a three-layer prompt structure:

```
Layer 1 (Safety):    "Do not generate harmful content. Stay within document
scope."
Layer 2 (Persona):   "You are DocMind AI, an expert enterprise document
analyst..."
Layer 3 (Task):      Document text + specific instruction
(summarise/compare/etc.)
```

This structure ensures consistent, professional, safe responses across all features.

6. Scalability and Innovativeness

Scalability

Horizontal Scaling:

- Flask + Gunicorn is stateless — multiple workers can run behind a load balancer
- Session data is stored in signed cookies (no server-side session storage needed)
- Rate limiting uses Redis in production (configurable via `REDIS_URL`) — works across workers

Database Scaling:

- SQLite for development → PostgreSQL for production (zero code change — just swap `DATABASE_URL`)
- `pool_pre_ping=True` keeps connections alive under load
- SQLAlchemy ORM supports migrations to any SQL database

AI Scaling:

- The `watsonx.ai` API is managed infrastructure — scales automatically on IBM's side
- Document text is stored in the database, not re-extracted on every AI call
- AI results (summaries, tags) are cached in the Document model to avoid redundant API calls

Storage Scaling:

- Upload folder is abstracted behind config — can be swapped for S3/Cloudinary with minimal service changes
- File metadata is decoupled from file storage (DB stores metadata, disk stores content)

Innovativeness

What makes DocMind AI genuinely innovative:

1. **Multi-modal document intelligence** — handles four file formats with a single unified AI interface. Most AI document tools support only PDFs.
2. **Persistent conversation memory** — conversation history is stored in the database per document, not just in the browser session. Users can come back days later and continue where they left off.
3. **Document comparison at scale** — comparing two documents for a human takes hours. DocMind AI does it in seconds with structured output (similarities, differences, key topics, summary).
4. **AI-generated tagging** — documents are automatically tagged with relevant keywords, making the document library self-organizing without any manual effort.
5. **Configurable AI personality** — the `AGENT_CONFIGURATION` dict in `config.py` lets non-developers customise tone, persona, response style, safety rules, and even swap the AI

model without touching any other code. This is a genuine enterprise feature.

- 6. **Zero-dependency dark mode** — the entire light/dark theme is implemented with CSS custom properties only — no JavaScript library, no separate stylesheet, no DOM manipulation beyond toggling a `data-theme` attribute. The theme preference is persisted per-user in the database.
- 7. **Prompt layering architecture** — safety + persona + task are layered in every prompt, meaning the AI behaves consistently whether the user is asking for a summary or chatting casually.

7. Contribution and Social Benefit

Who Benefits

Audience	How DocMind AI Helps
Students	Summarise textbooks, research papers, lecture notes in seconds. Generate interview questions from study material. Extract key concepts before exams.
Researchers	Quickly understand papers outside their domain. Compare methodologies across multiple papers. Extract keywords for literature review.
Legal Professionals	Parse long contracts and agreements. Extract deadlines, obligations, and key clauses. Compare two contract versions for differences.
HR & Recruiters	Summarise resumes and job descriptions. Extract key skills and qualifications. Compare candidate profiles.
Business Analysts	Extract insights from reports, financials, and strategy documents. Generate executive summaries for stakeholders.
Medical & Healthcare	Summarise clinical papers. Extract key findings. Explain complex medical language in simple terms.
Government & NGOs	Parse policy documents, legislation, and grant proposals. Make information accessible.
Teachers & Educators	Generate quiz questions from course material. Create lesson plan summaries.

Accessibility Impact

- The clean, simple interface requires no technical knowledge to use
- Supports multiple languages (English, Spanish, French, German, Hindi) in the AI responses

- Dark mode reduces eye strain for extended reading sessions
- Mobile-responsive design works on phones and tablets — not just desktops

Cost Reduction

For a small business or student, hiring a consultant to summarise or compare documents costs money and time. DocMind AI automates this at near-zero marginal cost, democratising access to document intelligence that was previously only available to large enterprises with dedicated knowledge management teams.

Privacy Respect

- All documents are stored per-user — no other user can access your documents
- Text extraction is done server-side — the raw file is not sent to any third party except the IBM watsonx.ai API (which is enterprise-grade and GDPR-compliant)
- Account deletion is complete and permanent — all files, text, conversations are removed immediately

8. Readiness for Deployment

Current Status

Component	Status	Notes
Flask application	Production-ready	Proper factory pattern, error handlers, logging
Database	Production-ready	SQLite for dev, PostgreSQL for production
Authentication	Production-ready	Rate limiting, secure cookies, PBKDF2 hashing
File uploads	Production-ready	UUID naming, type validation, size limits
IBM AI integration	Production-ready	Error handling, fallback messages, config-driven
WSGI server	Production-ready	Gunicorn with 2 workers, 120s timeout
Health check	Live	/health endpoint with DB ping
Error pages	Production-ready	Custom 404, 500, 429 for all user states
Security headers	Production-ready	ProxyFix, HttpOnly cookies, SameSite=Lax
Deployment configs	Ready	Procfile, render.yaml, railway.toml, vercel.json

What Is Needed Before a Production Launch

1. **Persistent file storage** — Render/Railway reset the filesystem on every deploy. Replace the local `uploads/` folder with Cloudinary or AWS S3 (2–3 hours of work).
2. **PostgreSQL database** — Replace SQLite with a managed PostgreSQL instance (Railway provides one free; Render provides 90-day free tier). Change `DATABASE_URL` only.
3. **Redis for rate limiting** — In-memory rate limiting doesn't work across multiple Gunicorn workers. Set `REDIS_URL` to a Redis instance URL.
4. **Real SECRET_KEY** — Generate a cryptographically secure random string:

```
python -c "import secrets; print(secrets.token_hex(32))"
```
5. **HTTPS** — All production platforms (Render, Railway, Heroku) provide HTTPS automatically. The app already sets `SESSION_COOKIE_SECURE=True` in `ProductionConfig`.

Deployment Commands

Render (recommended):

1. Push to GitHub
2. Connect repo on render.com
3. Set 6 environment variables in dashboard
4. Click Deploy

Railway:

```
railway login
railway link
railway up
```

Local:

```
cd docmind_ai
python -m venv venv
venv\Scripts\activate          # Windows
pip install -r requirements.txt
copy .env.example .env         # Then fill in your values
python run.py
```

9. Commercial and Market Viability

Market Size

The global Intelligent Document Processing (IDP) market was valued at **USD 1.49 billion in 2023** and is projected to reach **USD 7.56 billion by 2028** (CAGR of 38.4%). The rise of generative AI has accelerated adoption dramatically.

Competitive Landscape

Competitor	What They Offer	DocMind AI Advantage
ChatPDF	PDF-only Q&A	DocMind supports 4 formats + comparison + insights + teams
Adobe Acrobat AI	PDF summarisation	Not free, no comparison feature, vendor lock-in
Notion AI	Document workspace with AI	Requires all docs in Notion, expensive subscription
Microsoft Copilot	Office 365 document AI	Requires M365 subscription, Windows-centric
Humata	Document Q&A	No comparison, no multi-format, no open source

Business Models

Freemium SaaS:

- Free tier: 5 documents, 50 AI queries/month
- Pro (\$9.99/month): 100 documents, unlimited queries, conversation history export
- Team (\$29.99/month): 5 users, shared workspaces, admin dashboard
- Enterprise (custom): SSO, on-premises deployment, custom model fine-tuning

API as a Product:

- Expose the document processing pipeline as a REST API
- Target: developers building document-heavy applications
- Pricing: per-document or per-query billing

White-Label Licensing:

- Law firms, HR platforms, EdTech companies license DocMind AI as their own branded tool
- Revenue: one-time license fee + annual maintenance

Vertical Specialisation:

- DocMind Legal: contracts, clauses, jurisdiction analysis
- DocMind HR: resume screening, job description matching
- DocMind Research: paper summarisation, citation extraction

Revenue Potential

Assuming 1,000 paying Pro users at \$9.99/month:

- Monthly Revenue: \$9,990
- Annual Revenue: \$119,880

IBM watsonx.ai API costs (at scale) are well within margin — the platform is pay-per-token and designed for cost-efficient enterprise use.

10. Future Scope

Near-Term (3–6 months)

Feature	Description
Persistent file storage	Cloudinary or AWS S3 integration — uploaded files survive server restarts
PostgreSQL production DB	Migrate from SQLite for true production persistence and concurrent access
Email verification	Confirm email on registration to reduce spam accounts
Password reset by email	Forgot password flow via SMTP / SendGrid
Export summaries	Download AI-generated summaries as PDF or DOCX
Bulk document upload	Upload multiple files at once with a multi-file drag-drop zone
Document folders/workspaces	Organise documents into named collections

Medium-Term (6–12 months)

Feature	Description
IBM Granite models	Switch to IBM's own Granite foundation models when available in the project region — better suited to enterprise document tasks
Multi-document chat	Ask questions across multiple selected documents simultaneously
Streaming responses	Server-Sent Events (SSE) for real-time AI text streaming instead of waiting for full response
OCR for scanned PDFs	Use Tesseract or IBM Document Understanding to process scanned image PDFs
Team workspaces	Multiple users sharing a workspace with role-based access (viewer, editor, admin)
API rate dashboard	Show IBM watsonx.ai API usage stats and remaining quota to users
Citation mode	AI cites exact page number and paragraph in responses
Multi-language UI	i18n support — interface in Spanish, French, Hindi, etc.

Long-Term (12–24 months)

Feature	Description
Fine-tuned Granite model	Fine-tune IBM Granite on domain-specific document corpora (legal, medical, financial) for higher accuracy
Vector search (RAG)	Replace text truncation with a proper Retrieval-Augmented Generation pipeline using pgvector or Milvus — enables true full-document Q&A regardless of document length
Real-time collaboration	Multiple users editing annotations and summaries simultaneously (WebSockets)
Document version control	Track document revisions and compare versions
Audit trail	Compliance-grade logging of who accessed what document and when
On-premises deployment	Docker + Kubernetes deployment spec for enterprise clients who cannot use cloud
Mobile app	React Native app for uploading photos of physical documents
Automated insights pipeline	Auto-analyse every uploaded document immediately and send email digest
Integration marketplace	Connect DocMind AI to Slack, Notion, Google Drive, SharePoint

The Path to IBM Granite

The application is architecturally ready for IBM Granite. In `config.py`, changing:

```
"model_id": "meta-llama/llama-3-3-70b-instruct"
```

to:

```
"model_id": "ibm/granite-13b-instruct-v2"
```

is the **only code change required** to switch models. The entire service layer is model-agnostic by design.

IBM Granite models are purpose-built for enterprise document tasks:

- Granite-13B-Chat: optimised for document Q&A
 - Granite-20B-Multilingual: cross-language document understanding
 - Granite-8B-Code: for technical documentation and code analysis
-

11. Environment Variables Reference

Variable	Required	Default	Description
SECRET_KEY	Yes	change-me	Flask session signing key — must be long random string in production
FLASK_ENV	Yes	development	development or production
FLASK_DEBUG	No	1 (dev), 0 (prod)	Enable Flask debug mode
DATABASE_URL	No	SQLite local	Full database URL; use <code>sqlite:////tmp/docmind.db</code> on Render
IBM_API_KEY	Yes	(empty)	IBM Cloud API key from IAM
IBM_PROJECT_ID	Yes	(empty)	watsonx.ai project ID
IBM_WATSONX_URL	Yes	<code>https://us-south.ml.cloud.ibm.com</code>	watsonx.ai regional endpoint
UPLOAD_FOLDER	No	uploads	Path (relative to project root) for uploaded files
MAX_CONTENT_LENGTH	No	16777216 (16 MB)	Maximum file upload size in bytes
ALLOWED_EXTENSIONS	No	pdf, docx, pptx, txt	Comma-separated list of allowed file extensions
REDIS_URL	No	(memory://)	Redis URL for rate limiter storage in production
FLASK_HOST	No	127.0.0.1	Host to bind (local dev only)
FLASK_PORT	No	5000	Port to bind (local dev only)

12. API Endpoints Reference

All JSON API endpoints expect `Content-Type: application/json` and return JSON. All require a valid login session (@login_required).

POST /ai/chat/send

Request:

```
{
  "message": "What is the main argument of this document?",
  "doc_id": 42,
  "conv_id": 7
}
```

Response (success):

```
{
  "answer": "The document argues that...",
  "conv_id": 7,
  "conv_title": "What is the main argument..."
}
```

Response (error):

```
{ "error": "Document not found" }
```

POST /ai/summary/generate

Request:

```
{ "doc_id": 42, "style": "bullets" }
```

Response:

```
{
  "summary": "• Key point 1\n• Key point 2\n...",
  "doc_name": "report.pdf",
  "style": "bullets"
}
```

POST /ai/compare/run

Request:

```
{ "doc_id_a": 42, "doc_id_b": 43 }
```

Response:

```
{
  "similarities": "Both documents discuss...",
  "differences": "Document A focuses on X while Document B...",
  "key_topics": "Revenue, Growth, Strategy",
  "summary": "Overall, the documents share...",
  "raw": "(full AI response text)"
}
```

POST /ai/insights/generate

Request:

```
{ "doc_id": 42 }
```

Response:

```
{
  "keywords": ["machine learning", "neural networks", "training data"],
  "topics": ["Artificial Intelligence", "Model Training"],
  "tags": ["AI", "Technical", "Research"],
  "one_line_summary": "A technical paper on deep learning model optimisation.",
  "reading_time": 12,
  "word_count": 2350,
  "page_count": 8,
  "doc_name": "paper.pdf"
}
```

GET /documents/api/list

Response:

```
[
  { "id": 42, "name": "report.pdf", "type": "pdf", "size": "2.3 MB" },
  { "id": 43, "name": "notes.docx", "type": "docx", "size": "145.0 KB" }
]
```

GET /health

Response (healthy):

```
{ "status": "ok", "database": true }
```

HTTP status: 200 (healthy) or 503 (database unreachable)

13. Security Architecture

Authentication

- Passwords hashed with PBKDF2-SHA256 (Werkzeug) — industry standard, salted
- Login sessions use signed HTTP-only cookies — cannot be read by JavaScript
- SameSite=Lax cookie policy prevents CSRF on most requests
- SameSite=Strict + Secure flag enabled in production config
- 30-day session lifetime with "Remember Me" support

Rate Limiting

- Login: 20 attempts per hour per IP → prevents brute-force attacks
- Register: 10 attempts per hour per IP → prevents account spam
- Global default: 200 requests/day, 50/hour per IP
- Backed by Redis in production for accuracy across workers

File Upload Security

- Files are renamed to UUID hex strings on disk — original filenames never used for storage

- File type validated by extension (whitelist approach)
- File size limited to 16 MB by Flask's `MAX_CONTENT_LENGTH`
- Files stored outside the web-accessible `static/` directory

Access Control

- Every document and conversation route checks `user_id=current_user.id`
- Users can only access their own data — horizontal privilege escalation is prevented
- `@login_required` decorator on all authenticated routes

API Key Management

- IBM API key is loaded from `.env` → never hardcoded
- `.env` is in `.gitignore` — never committed to version control
- `.env.example` contains only placeholder values

Input Safety

- AI prompts include a safety preamble on every call
- Jinja2 auto-escaping prevents XSS in rendered templates
- Flask-WTF provides CSRF token protection on forms